

Chapitre 3: Collection d'objets : les chaînes de caractères, les listes, les tuples, les ensembles, les dictionnaires, les opérations et méthodes associées.

Les chaînes de caractères

Le type des chaînes de caractères, *string* en anglais et dans Python, est celui permettant de représenter des textes. On considère dans un premier temps des textes élémentaires, ceux composés d'une unique lettre ; on les appelle les *caractères*.

En Python, les caractères peuvent être n'importe quelle lettre de l'alphabet, mais aussi des symboles, comme les signes de ponctuation. Une chaîne de caractères est une suite finie de caractères consécutifs, qu'on note entre apostrophes ou guillemets :

'Ceci est une chaine'

La chaîne vide se note "" ou ""

Accès à un caractère:

On peut stocker une chaîne dans une variable :

s = 'Bonjour'

et accéder à chacun des caractères à l'aide de la construction s[i] :

```
>>>s[2]
```

'n'

Comparaison:

```
>>> "bonjour" == "bonne journée"
```

False

Les chaînes de caractères

les chaînes sont immuables

```
>>> s[0] = 'A'
```

TypeError: 'string' object does not support item assignment

Concaténation: On concatène deux chaînes à l'aide de l'opérateur + :

```
>>>'Bonjour ' + 'lecteur !'
```

```
'Bonjour lecteur !'
```

Longueur: On utilise len pour obtenir la longueur d'une chaîne :

```
>>>len('Bonjour')
```

```
7
```

Sous-chaînes

Un ensemble de caractères consécutifs à l'intérieur d'une chaîne s'appelle une sous-chaîne. Ainsi, 'lecteur' ou 'jour lec' sont des sous-chaînes de 'Bonjour lecteur !'.

Pour extraire une sous-chaîne de s, on écrit s[i:j] où i est l'indice du premier caractère de la sous-chaîne et j est l'indice du dernier caractère plus un.

Exemples :

```
>>>s = 'Bonjour lecteur !'
```

```
>>>s[0:7]    # de s[0] à s[6]
```

```
'Bonjour'
```

Les chaines de caractères

```
>>> 'j' in 'Bonjour lecteur !'
```

```
True
```

Il est à noter qu'il est également possible de tester la présence d'une sous-chaîne dans une chaîne avec la même construction :

```
>>>'lecteur' in 'Bonjour lecteur !'
```

```
True
```

```
>>>'Bjr' in 'Bonjour lecteur !'
```

```
False
```

Conversion vers des types simples:

On peut convertir une valeur d'un type simple vers une chaîne de caractères à l'aide de la construction `str(e)`. La chaîne obtenue est la même que celle que Python affiche par évaluation de `e` :

```
>>>str(1.2)
```

```
'1.2'
```

Il est possible de reconverter une telle chaîne vers une valeur d'un type simple :

```
>>>int('123')
```

```
123
```

```
>>>float('1.2')
```

```
1.2
```

```
>>>bool('True')
```

```
True
```

```
>>> p='abcde'; q=p*3
```

```
q='abcdeabcdeabcde'
```

Code ASCII en Python:

`ord('A')=65` et `chr(65)='A'`

`ord('0')=48` et `chr(48)='0'`

Les chaines de caractères

```
>>> x="Bonjour aux étudiants"
```

```
>>> y=x.split()
```

```
➔y= ['Bonjour', 'aux', 'étudiants' ]
```

```
>>> n=x.count('o') # → 2
```

```
>>> x.endswith("ants") # → True
```

```
>>> x.startswith("Bon") # → True
```

```
>>>y =x.lower()# ou x.casefold() qui fonctionne même pour des caractères  
#spéciaux comme "\u00DF" qui correspond à "ß"
```

```
#char = "\u00DF" ;
```

```
#print(char.lower()) # → "ß"
```

```
#print(char.casefold()) # → "ss"
```

```
➔y= "bonjour aux étudiants"
```

```
>>> y=x.upper()
```

```
➔'BONJOUR AUX ETUDIANTS'
```

```
>>>x.find("jour")
```

```
➔3
```

```
>>> x.index("jour")
```

```
➔3
```

Les chaines de caractères

```
>>> x="Bonjour mes chers étudiants"
```

```
>>> x.islower()
```

```
False
```

```
>>> x.isupper()
```

```
False
```

```
>>> x.isdigit()
```

```
False
```

```
>>> x.isalpha()
```

```
False
```

```
>>> x.isalnum()
```

```
False
```

```
>>> y=x.replace(' ','')
```

```
>>> y
```

```
'Bonjourmeschersétudiants'
```

```
>>> y.isalnum()
```

```
True
```

```
>>> chaine = 'Aujourd'hui'
```

```
>>> chaine = 'Aujourd\'hui'
```

```
>>> print (chaine)
```

```
>>> chaine = "Aujourd'hui"
```

```
>>> print (chaine)
```

```
>>> chaine = 'Première ligne\nDeuxième ligne'
```

```
>>> print(chaine)
```

```
>>> X="Pa"
```

```
>>> Y=X*2
```

```
➔ "PaPa"
```

```
>>> X="Bon"
```

```
>>> Y=X*2
```

```
➔ "BonBon"
```

Les chaines de caractères

```
>>> " ".isspace()
→ True
>>> "Bonjour Monsieur".isspace()
→ False
>>> "      ".isspace()
→ True
>>> "\t".isspace()
→ True
>>> "Titre".istitle()
→ True
>>> "TitrE".istitle()
→ False
>>> "Titre de mon site".istitle()
→ False
>>> "Titre De Mon Site".istitle()
→ True
>>> "***".join(["Master", "Data", "Science"])
→ "Master***Data***Science"
→ >>> "Bonjour".replace("jour", " week-end")
→ "Bon week-end")
```

Les chaines de caractères

```
texte ="2025"+chr(0x00B3)
```

```
print(texte) # → 2025³
```

```
texte ="2025"+"   B3"
```

```
print(texte) # → 2025³
```

```
print(texte.isdecimal()) # →False
```

```
print(texte.isdigit()) # →True
```

```
texte = "\u2163" # texte est le chiffre  
#romain "IV"
```

```
print(texte.isdecimal()) # →False
```

```
print(texte.isdigit()) # →False
```

```
print(texte.isnumeric()) # →True
```

```
a="\u00BD" #     : 1/2
```

```
b="\u00BE" #     :    
```

```
print(a.isdecimal()) # →False
```

```
print(a.isdigit()) # →False
```

```
print(a.isnumeric()) # →True
```

- **isdecimal()** : v  rifie si tous les caract  res sont des chiffres d  cimaux (0–9) ; il ne reconna  t ni exposants, ni fractions, ni chiffres romains.

- **isdigit()** : est plus large : il inclut les chiffres d  cimaux et les caract  res Unicode repr  sentant des chiffres comme les exposants ou indices, mais pas les fractions ou chiffres romains.

- **isnumeric()** : est le plus complet : il reconna  t tous les chiffres, exposants, indices, fractions Unicode et chiffres romains, permettant ainsi de d  tecter tout symbole num  rique Unicode.

Les chaines de caractères

```
>>>"Master\nBig\nData\nIA".splitlines()
```

```
➔['Master', 'Big', 'Data','IA']
```

```
>>> x="bonjour amine"
```

```
>>> x.capitalize()
```

```
➔'Bonjour amine'
```

```
>>> e="\n"
```

```
>>> e.isprintable()
```

```
➔False
```

```
>>> x='bon'
```

```
>>> x.isprintable()
```

```
➔True
```

```
>>> x='bon'
```

```
>>> x.isprintable()
```

```
➔True
```

```
>>> y="c'est une variable 100%"
```

```
>>> y.isidentifier()
```

```
➔False
```

```
>>> x="Bonjour Chers Etudiants"
```

```
>>> x.swapcase()
```

```
➔'bONJOUR cHERS eTUDIANTS'
```

```
>>> x="ceCi n'eSt pas BOn"
```

```
>>> x.title()
```

```
➔"Ceci N'Est Pas Bon"
```

Les chaines de caractères

```
x=" bonjour à tous "  
a=x.ljust(40,'*')  
print(a)  
print(len(a))  
➔
```

bonjour à tous *****

40

```
x=" bonjour à tous "  
a=x.center(40,'*'); print(a); print(len(a))  
➔          ***** bonjour à tous *****  
          40
```

```
x=" bonjour à tous "  
a=x.lstrip() #efface les espaces de début  
print(a)  
➔  
bonjour à tous
```

```
x="bonne***année universitaire à tous"  
a=x.partition('***')  
print(a)  
➔ ('bonne', '***', 'année universitaire à tous')
```

Les chaines de caractères

```
x="bonne année universitaire à tous"
```

```
a=x.rjust(40,'*')
```

```
print(a)
```

```
➔ *****bonne année universitaire à tous
```

```
>>> a
'      bonne          année          universitaire à tous'
>>> b
'bonne          année          universitaire à tous      '
>>> c
'bonne          année          universitaire à tous'
>>>
```

```
<tmp 1>
1 x="      bonne          année          universitaire à tous      "
2 a=x.rstrip(); print(a)
3 b=x.lstrip(); print(a)
4 c=x.strip(); print(a)
```

```
>>> x='bon courage. bonne journée'
```

```
>>> x.find('bon')
```

```
0
```

```
>>> x.rfind('bon')
```

```
13
```

Les chaines de caractères

str.expandtabs() est une méthode qui permet de **remplacer les tabulations "\t" par des espaces** dans une chaîne. Cela est utile pour afficher du texte de façon alignée.

Données:

```
texte = "Nom\tPrénom\tÂge\tVille\n"
texte += "Mohamed\tAli\t25\tCasablanca\n"
texte += "Fatima\tZahra\t30\tRabat\n"
texte += "Youssef\tOmar\t22\tMarrakech\n"
```

Affichage avant expandtabs:

```
print("Avant expandtabs :")
print(texte)
```

Affichage après expandtabs avec

tabsize=10: un bon alignement

```
print("Après expandtabs :")
print(texte.expandtabs(10))
```

Avant expandtabs :

Nom	Prénom	Âge	Ville
Mohamed	Ali	25	Casablanca
Fatima	Zahra	30	Rabat
Youssef	Omar	22	Marrakech

Après expandtabs :

Nom	Prénom	Âge	Ville
Mohamed	Ali	25	Casablanca
Fatima	Zahra	30	Rabat
Youssef	Omar	22	Marrakech

Les chaînes de caractères

`str.rfind()` est une méthode qui permet de **chercher la dernière occurrence d'une sous-chaîne dans une chaîne** et de renvoyer l'indice de départ.
Si la sous-chaîne n'est pas trouvée, elle renvoie **-1**.

```
>>> ch="bonjour Mohammed ouidadi"  
>>> ch.rfind("o") # donne 17  
>>> ch.rfind("o",5) # donne 17  
>>> ch.rfind("o",5,10) # donne 9
```

`str.partition()` est une méthode pratique pour **découper une chaîne en trois parties** autour d'un séparateur spécifique.

```
>>> ch="Mohammed ouidadi casablanca"  
>>> ch.partition("ouidadi")  
( 'Mohammed ', 'ouidadi', ' casablanca' )  
  
>>> ch.split("ouidadi")  
[ 'Mohammed ', ' casablanca' ]
```

Les chaines de caractères

rfind(sub) : retourne l'indice de la dernière occurrence de sub. Si sub n'existe pas → -1.

rindex(sub) : identique à rfind mais lève une exception si sub n'est pas trouvé.

rpartition(sep) : découpe la chaîne en trois parties autour de la dernière occurrence de

sep : avant : tout avant sep : le séparateur après : tout après

```
texte = "bonsoir Mohamed, bonne nuit Mohamed A+"
```

```
try:
```

```
    index_rindex = texte.rindex("Mohamed")
```

```
    print("rindex :", index_rindex)
```

```
except ValueError:
```

```
    print("rindex : substring non trouvée")
```

```
→ rindex : 28
```

```
print(texte.rfind("Mohamed"))
```

```
→ 28
```

```
L = texte.rpartition("Mohamed")
```

```
print(L)
```

```
→ ('bonsoir Mohamed, bonne nuit ', 'Mohamed', ' A+')
```

Les chaines de caractères

str.maketrans() est utilisé pour créer une **table de traduction** pour la méthode **str.translate()**, qui permet de **remplacer ou supprimer des caractères** dans une chaîne de manière efficace.

Syntaxe : str.maketrans(x, y, z)

- x : chaîne contenant les caractères à remplacer
- y : chaîne contenant les caractères correspondants de remplacement (doit être de même longueur que x)
- z (optionnel) : chaîne des caractères à supprimer

Remarques:

- 1) str.maketrans() ne fait rien tout seul ; il crée la table pour translate().**
- 2) Efficace pour nettoyer du texte ou remplacer plusieurs caractères à la fois.**
- 3) Utilisé souvent en prétraitement de texte en Data Science** (suppression de ponctuation, accents, etc.).

```
ch="ok! ça marche saïd ?"  
t=ch.maketrans("çï","ci","!?")  
texte_netoye = ch.translate(t)  
print(texte_netoye)
```

→ ok ca marche said

Les chaînes de caractères

En **sciences de données**, le **codage UTF-8** permet de manipuler souvent des textes multilingues (français, arabe, ...).

En **Python 2**, les fichiers source étaient supposés utiliser ASCII par défaut. Il fallait alors écrire en début de fichier :

```
# -*- coding: utf-8 -*-
```

pour indiquer que le fichier source était en UTF-8, surtout si tu utilisais des caractères spéciaux dans le code (é, ç, ß, etc.).

En Python 3

UTF-8 est l'encodage par défaut pour les fichiers source.

On peut écrire directement :

```
texte = "Bonjour, ça va ? مرحبا ؟" ;  
print(texte)
```

Les chaines de caractères

Chaîne avec accents et un mot arabe

```
texte = "Salam, ça va ?مرحبا "
```

#Encodage en UTF-8 (recommandé pour le multilingue)

```
utf8_bytes = texte.encode("utf-8")
```

```
print("UTF-8 bytes :", utf8_bytes)
```

```
# → b'Salam, \xc3\xa7a va ? \xd9\x85\xd8\xb1\xd8\xad\xd8\xa8\xd8\xa7'
```

Décodage pour revenir à str

```
texte_decode = utf8_bytes.decode("utf-8")
```

```
print("Décodé :", texte_decode)
```

```
# → Salam, ça va ?مرحبا
```

#Encodage en ASCII avec remplacement des caractères non-ASCII

```
ascii_bytes = texte.encode("ascii", errors="replace")
```

```
print("ASCII avec remplacement :", ascii_bytes)
```

```
# → b'Salam, ?a va ? ????'
```

certains logiciels anciens ou très spécialisés ne supportent pas UTF-8 et ne gèrent que l'ASCII

Encodage en ASCII en ignorant les caractères non-ASCII

```
ascii_bytes_ignore = texte.encode("ascii", errors="ignore")
```

```
print("ASCII en ignorant :", ascii_bytes_ignore)
```

```
# → b'Salam, a va ? '
```

Les chaines de caractères

La méthode **str.format(*args, **kwargs)** permet de **formater une chaîne en insérant des valeurs** passées sous forme de **arguments positionnels (*args)** ou **arguments nommés (**kwargs)**.

1) Arguments positionnels (*args)

Les {} sont remplacés par les arguments dans l'ordre

```
texte = "Je m'appelle {} et j'ai {} ans.".format("Mohamed", 25)  
print(texte)
```

→ Je m'appelle Mohamed et j'ai 25 ans.

On peut aussi utiliser des **indices** pour réorganiser :

```
texte = "Nom: {0}, Age: {1}, Encore Nom: {0}".format("Mohamed", 25)  
print(texte)
```

→ Nom: Mohamed, Age: 25, Encore Nom: Mohamed

Les chaines de caractères

2) Arguments nommés (**kwargs)

```
texte = "Je m'appelle {nom} et j'ai {age} ans.".format(nom="Mohamed", age=25)  
print(texte)
```

→ Je m'appelle Mohamed et j'ai 25 ans.

On peut combiner **positionnel + nommé** :

```
texte = "Nom: {0}, Age: {age}".format("Mohamed", age=25)  
print(texte)
```

→ Nom: Mohamed, Age: 25

3) Version avec dictionnaire (sera présenté ultérieurement dans ce chapitre)

```
info = {"nom": "Mohamed", "age": 25}  
texte = "Je m'appelle {nom} et j'ai {age} ans.".format(**info)  
print(texte)
```

→ Je m'appelle Mohamed et j'ai 25 ans.

Les chaines de caractères

4) **str.format_map()** est une méthode similaire à **str.format()**, mais elle **prend directement un dictionnaire** (ou tout objet qui se comporte comme un mapping) pour remplacer les variables dans une chaîne.

Syntaxe : str.format_map(mapping)

Exemple :

```
personne = {"nom": "Mohamed", "age": 25}
texte = "Je m'appelle {nom} et j'ai {age} ans.".format_map(personne)
print(texte)
# → Je m'appelle Mohamed et j'ai 25 ans.
```



```
personne = {"nom": "Mohamed", "age": 25}
texte = "Je m'appelle {nom} et j'ai {age} ans.".format(**personne)
print(texte)
# → Je m'appelle Mohamed et j'ai 25 ans.
```

Les chaînes de caractères

str.zfill() est une méthode qui **remplit une chaîne avec des zéros à gauche** pour atteindre une longueur donnée.

Très utilisé pour **formater des numéros, IDs, dates, codes postaux**, etc.

```
jour = "3"  
mois = "9"  
annee = "2025"  
date = jour.zfill(2)+"/"+mois.zfill(2)+"/"+annee.zfill(4)  
print(date)
```

→ 03/09/2025

Exercice d'application:

1) Que pensez-vous du code suivant et quel est son résultat :

```
ch="Bonjour DSBD-IA"  
for i in range(len(ch)):  
    ch[i]="*"  
print(ch)
```

2) Que pensez-vous du code suivant et quel est son résultat :

```
def rendreMajuscule(ch):  
    ch=ch.upper()  
#####  
A="Bonjour DSBD-IA"  
B= rendreMajuscule(A)  
print(B)
```